

Network Programming

(Day 4)

Yunmin Go

School of AICE

Handong Global University

Network Programming (Day 4): roadmap

- Multi-Thread Programming #1
- Multi-Thread Programming #2
- Web Server Implementation

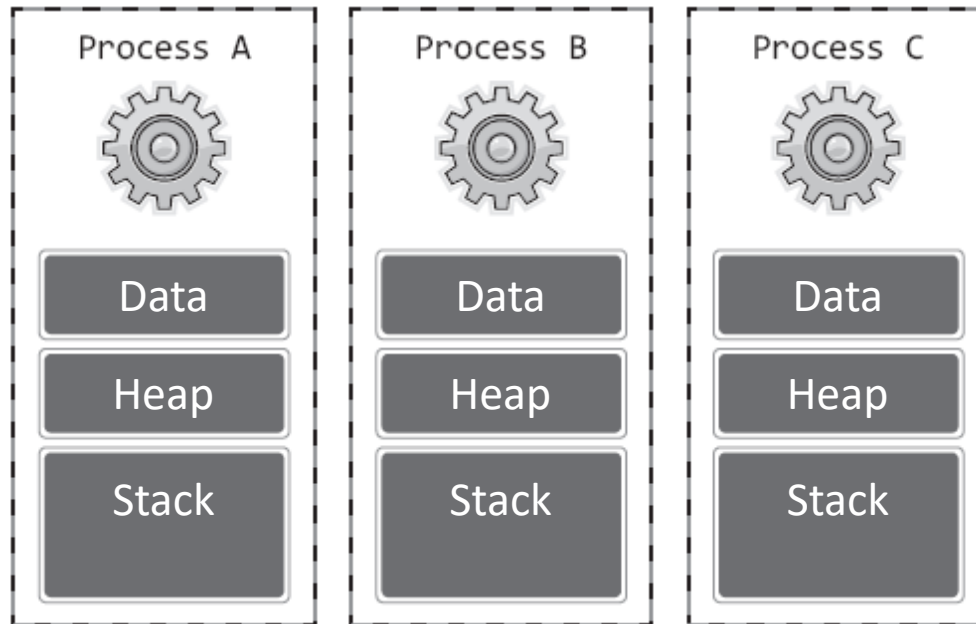
Thread

- Light weight process

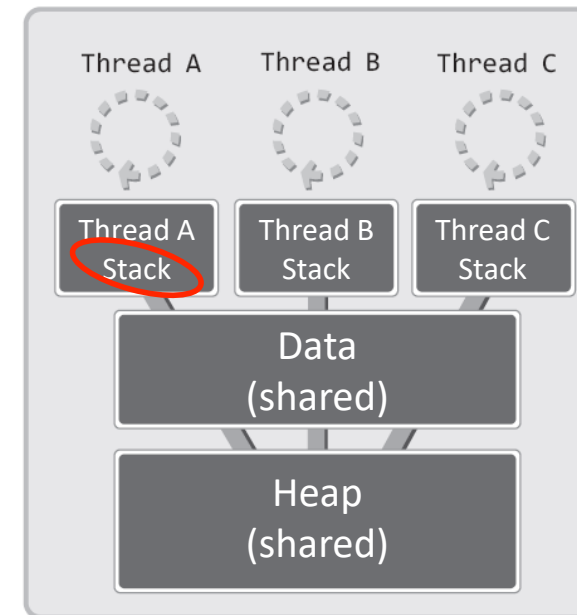
- Thread requires fewer system resources than process.
- All threads within a process share the same address space except for the stack space.
- Inter-thread communication is more efficient and easier to use than inter-process communication

Thread vs. Process

- Processes maintain separate memory space between processes
- Threads share the same memory space except for stack



<Process>



<Thread>

Pthread

- Pthread is a standardized thread programming interface specified by the IEEE POSIX (portable operating system interface) in 1995.
- Pthreads are defined as a set of C language programming types and procedure calls, implemented with a pthread.h header/include file and a thread library.

Pthread API

- Thread management `pthread_~~`
 - The functions work directly on threads – creating, terminating, joining, etc.
- Mutexes
 - Provide for creating, destroying, locking, and unlocking mutexes
- Semaphores
 - Provide for creating, destroying, waiting, and posting on semaphores
- Conditional variables
 - Include functions to create, destroy, wait, and signal based on specified variable values.

Thread Creation

```
#include <pthread.h>
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
```

Create a new thread

- Starts a new thread in the calling process
- *thread*: new thread ID on success
- *attr*: to set thread attribute. NULL for default value.
- *start_routine*: the routine that the thread will execute when it is created
- *arg*: argument to be passed to
- Return value
 - Success: 0
 - Error: error number

Thread Join

```
#include <pthread.h>
int pthread_join(pthread_t thread, void **retval);
```

Join with a terminated thread

- It blocks the calling thread until the specified thread returns
- When a parent thread returns, all the child thread are terminated
- *thread*: specified thread to terminate
- *retval*: return value from the terminated thread (a pointer returned by reference)
- Return value
 - Success: 0
 - Error: error number

Example: thread1.c

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum;
void *runner(void *param);

int main(int argc, char *argv[])
{
    pthread_t tid;

    // create the thread
    pthread_create(&tid, NULL, runner, argv[1]);

    // now wait for the thread to exit
    pthread_join(tid, NULL);

    printf("sum = %d\n", sum);

    return 0;
}
```

```
// thread will execute in this function
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0); // or return NULL
}
```

Results

```
[gym@localhost practice10]$ gcc thread1.c -o thread1 -lpthread
[gym@localhost practice10]$ ./thread1 10
sum = 55
[gym@localhost practice10]$ ./thread1 5
sum = 15
```

Example: thread2.c

```
#include <stdio.h>
#include <pthread.h>
void * thread_summation(void * arg);

int sum=0;

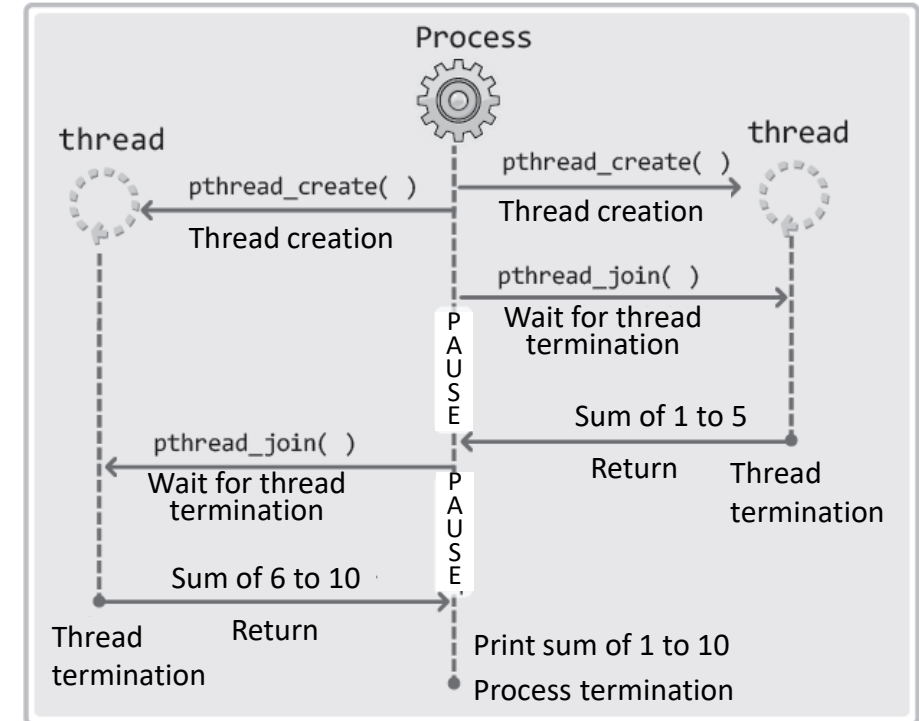
int main(int argc, char *argv[])
{
    pthread_t id_t1, id_t2;
    int range1[] = {1, 5};
    int range2[] = {6, 10};

    pthread_create(&id_t1, NULL, thread_summation, (void *)range1);
    pthread_create(&id_t2, NULL, thread_summation, (void *)range2);

    pthread_join(id_t1, NULL);
    pthread_join(id_t2, NULL);
    printf("result: %d \n", sum);
    return 0;
}
```

```
void * thread_summation(void * arg)
{
    int start = ((int*)arg)[0];
    int end = ((int*)arg)[1];

    while (start <= end)
    {
        sum+=start;
        start++;
    }
    return NULL;
}
```



Results

```
[gym@localhost practice10]$ gcc thread2.c -o thread2 -lpthread
[gym@localhost practice10]$ ./thread2
result: 55
```

Thread Detach

```
#include <pthread.h>
int pthread_detach(pthread_t thread);
```

Detach a thread

- Marks the thread identified by *thread* as detached
- When a detached thread terminates, its resources are automatically released back to the system without the need for another thread to join with the terminated thread.
- Non-blocking function
- *thread*: specified thread to terminate
- Return value
 - Success: 0
 - Error: error number

Thread Cancel

```
#include <pthread.h>
int pthread_cancel(pthread_t thread);
```

Send a cancellation request to a thread

- This function sends a cancellation request to the thread
- Whether and when the target thread reacts to the cancellation request depends on two attributes that are under the control of that thread: its cancelability state and type.
- Cancelability state is determined by `pthread_setcancelstate()`
- Cancellation type is determined by `pthread_setcanceltype()`
- `thread`: thread ID to cancel
- Return value
- Success: 0
- Error: nonzero error number

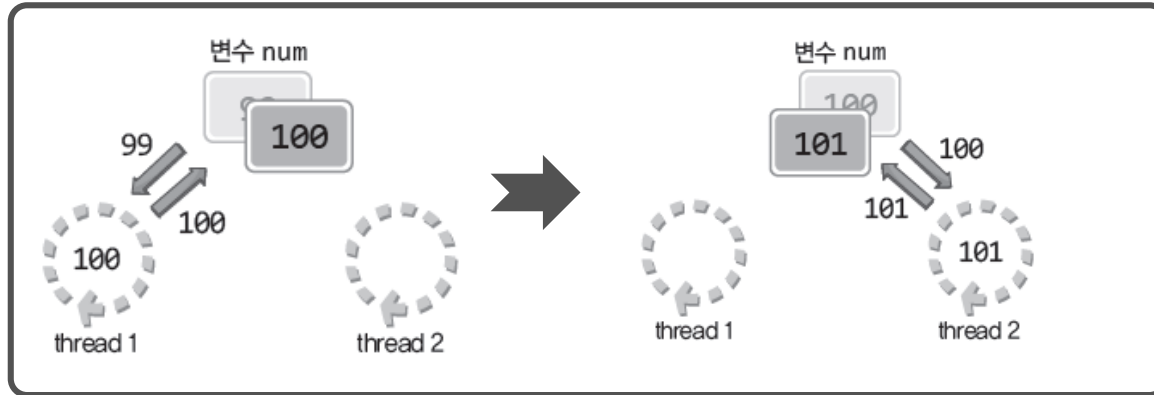
Network Programming (Day 4): roadmap

- Multi-Thread Programming #1
- **Multi-Thread Programming #2**
- Web Server Implementation

Synchronization

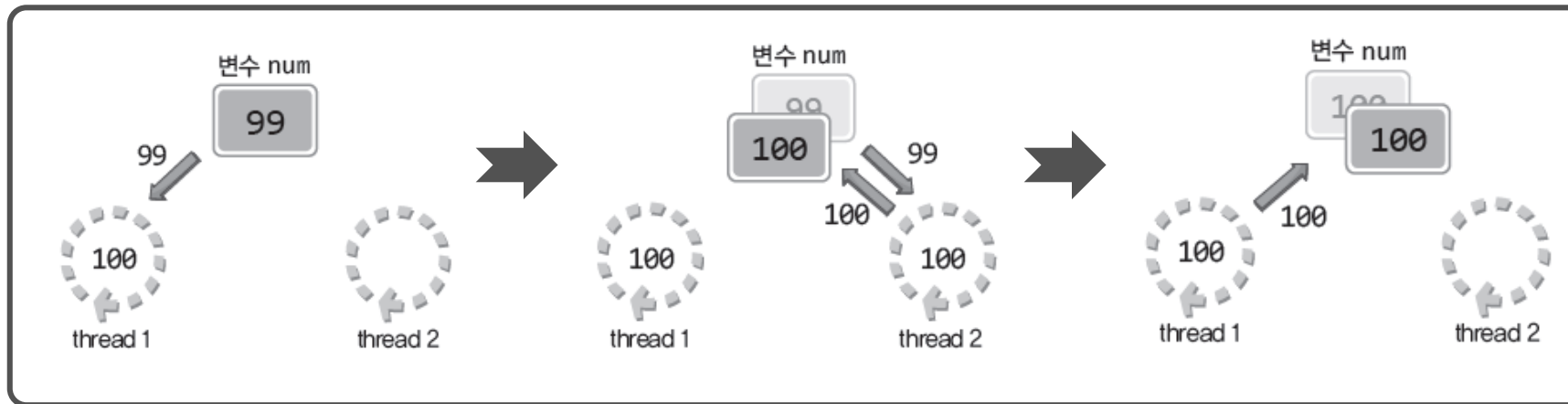
- Synchronization problem
 - When more than two thread access a shared data, only one thread can access the shared data and other threads should not access it.
- Typical synchronization methods
 - Mutex
 - Semaphore

Synchronization problem



Normal

If you access the variable num sequentially, **no** problem occurs.



Abnormal

Thread 1 and thread 2 each increase by 1, but the variable num increase by 1 only.

Example: critical.c

```
#include <stdio.h>
#include <pthread.h>
#define NUM_THREAD      10

void *thread_inc(void * arg);
void *thread_des(void * arg);
long long num = 0;

int main(int argc, char *argv[])
{
    pthread_t thread_id[NUM_THREAD];
    int i;

    printf("sizeof long long: %d \n", sizeof(long long));
    for (i = 0; i < NUM_THREAD; i++)
    {
        if (i % 2)
            pthread_create(&(thread_id[i]), NULL, thread_inc, NULL);
        else
            pthread_create(&(thread_id[i]), NULL, thread_des, NULL);
    }

    for (i = 0; i < NUM_THREAD; i++)
        pthread_join(thread_id[i], NULL);

    printf("result: %lld \n", num);
    return 0;
}
```

```
void *thread_inc(void * arg)
{
    int i;
    for (i = 0; i < 100000; i++)
        num += 1;
    return NULL;
}

void *thread_des(void * arg)
{
    int i;
    for (i = 0; i < 100000; i++)
        num -= 1;
    return NULL;
}
```

Results

```
[gym@localhost practice11]$ ./critical
sizeof long long: 8
result: 37873
[gym@localhost practice11]$ ./critical
sizeof long long: 8
result: 3084
[gym@localhost practice11]$ ./critical
sizeof long long: 8
result: -48707
[gym@localhost practice11]$ ./critical
sizeof long long: 8
result: -33140
[gym@localhost practice11]$ ./critical
sizeof long long: 8
result: -73883
```

Different results each time you run it

Mutex

- A mutex variable acts like a “lock” protecting access to a shared data resource.
- Only one thread can lock (or own) a mutex variable at any given time. Thus, even if several threads try to lock a mutex only one thread will be successful.
- No other thread can own that mutex until the owning thread unlocks that mutex. Threads must “take turns” accessing protected data.

Mutex functions

```
include <pthread.h>
int pthread_mutex_init(pthread_mutex_t *restrict mutex,
                        const pthread_mutexattr_t *restrict attr);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- Mutex variable (mutex) must be initialized by a `pthread_mutex_init()` before it can be used.
- *mutex*: mutex variable
- *attr*: NULL for default
- The `pthread_mutex_lock()` function is used by a thread to acquire a lock on the specified mutex variable. The thread blocks if the mutex is already locked by another thread
- The `pthread_mutex_unlock()` will unlock a mutex if called by the owning thread.

Example: mutex.c

```
#define NUM_THREAD      10
void *thread_inc(void * arg);
void *thread_des(void * arg);
long long num = 0;
pthread_mutex_t mutex;

int main(int argc, char *argv[])
{
    pthread_t thread_id[NUM_THREAD];
    int i;

    pthread_mutex_init(&mutex, NULL);

    for (i = 0; i < NUM_THREAD; i++)
    {
        if (i % 2)
            pthread_create(&(thread_id[i]), NULL, thread_inc, NULL);
        else
            pthread_create(&(thread_id[i]), NULL, thread_des, NULL);
    }

    for(i = 0; i < NUM_THREAD; i++)
        pthread_join(thread_id[i], NULL);

    printf("result: %lld \n", num);
    pthread_mutex_destroy(&mutex);
    return 0;
}
```

```
void *thread_inc(void * arg)
{
    int i;
    pthread_mutex_lock(&mutex);
    for (i = 0; i < 100000; i++)
        num += 1;
    pthread_mutex_unlock(&mutex);
    return NULL;
}

void *thread_des(void * arg)
{
    int i;
    for (i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&mutex);
        num -= 1;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

Results

```
[gym@localhost practice11]$ ./mutex
result: 0
[gym@localhost practice11]$ ./mutex
result: 0
[gym@localhost practice11]$ ./mutex
result: 0
[gym@localhost practice11]$ ./mutex
result: 0
[gym@localhost practice11]$ ./mutex
result: 0
```

Semaphore

- Semaphore are not defined in the POSIX.4a (pthread) specifications, but they are included in the POISX.4 (realtime extension) specifications.
- A semaphore has a type `sem_t`.
- A semaphore has a value of 0 or positive integer.
 - A binary semaphore descriptor has a value of 0 or 1.

Semaphore: `sem_init()` & `sem_wait()`

```
#include <semaphore.h>
int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_wait(sem_t *sem);
```

- `sem_init()`: initialize a semaphore
 - *sem*: a semaphore initialized to 'value'
 - *pshared*: if '0', only one process will use it. If nonzero several threads can share the semaphore.
 - *value*: the initial value of a semaphore value
 - Return
 - Success: 0
 - Error: -1
- `sem_wait()`: lock (decrements) a semaphore
 - If the semaphore value is positive, it decrements the value by 1.
 - If the semaphore value is zero, it waits until a thread unlocks the semaphore.
 - Return
 - Success: 0
 - Error: -1

Semaphore: `sem_post()` & `sem_destroy()`

```
#include <semaphore.h>
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);
```

- `sem_post()`: **unlock** (increments) a semaphore
 - Increments the semaphore by 1
 - Return
 - Success: 0
 - Error: -1
- `sem_destroy()`: destroy the semaphore *sem*
 - Return
 - Success: 0
 - Error: -1

Example: semaphore.c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

void *read(void * arg);
void *accu(void * arg);
static sem_t sem_one;
static sem_t sem_two;
static int num;

int main(int argc, char *argv[])
{
    pthread_t id_t1, id_t2;
    sem_init(&sem_one, 0, 0);
    sem_init(&sem_two, 0, 1);

    pthread_create(&id_t1, NULL, read, NULL);
    pthread_create(&id_t2, NULL, accu, NULL);

    pthread_join(id_t1, NULL);
    pthread_join(id_t2, NULL);

    sem_destroy(&sem_one);
    sem_destroy(&sem_two);
    return 0;
}
```

```
void *read(void * arg)
{
    int i;
    for (i = 0; i < 5; i++)
    {
        fputs("Input num: ", stdout);

        sem_wait(&sem_two);
        scanf("%d", &num);
        sem_post(&sem_one);
    }
    return NULL;
}

void *accu(void * arg)
{
    int sum=0, i;
    for (i = 0; i < 5; i++)
    {
        sem_wait(&sem_one);
        sum += num;
        sem_post(&sem_two);
    }
    printf("Result: %d \n", sum);
    return NULL;
}
```

Results

```
[gym@localhost practice11]$ ./semaphore
Input num: 10
Input num: 20
Input num: 30
Input num: 40
Input num: 50
Result: 150
```

Multi-thread Server: chat_serv.c

```
#define BUF_SIZE 100
#define MAX_CLNT 256
...
int clnt_cnt = 0;
int clnt_socks[MAX_CLNT];
pthread_mutex_t mutx;

int main(int argc, char *argv[])
{
    ...
    pthread_t t_id;
    ...
    pthread_mutex_init(&mutx, NULL);
    serv_sock = socket(PF_INET, SOCK_STREAM, 0);
    ...
    while (1)
    {
        clnt_adr_sz = sizeof(clnt_adr);
        clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_adr, &clnt_adr_sz);

        pthread_mutex_lock(&mutx);
        clnt_socks[clnt_cnt++] = clnt_sock;
        pthread_mutex_unlock(&mutx);

        pthread_create(&t_id, NULL, handle_clnt, (void*)&clnt_sock);
        pthread_detach(t_id);
        printf("Connected client IP: %s \n", inet_ntoa(clnt_adr.sin_addr));
    }
    ...
}
```

```
void *handle_clnt(void * arg)
{
    int clnt_sock = *((int*)arg);
    int str_len = 0, i;
    char msg[BUF_SIZE];

    while ((str_len = read(clnt_sock, msg, sizeof(msg))) != 0)
        send_msg(msg, str_len);

    pthread_mutex_lock(&mutx);
    for (i = 0; i < clnt_cnt; i++) // remove disconnected client
    {
        if (clnt_sock == clnt_socks[i])
        {
            while (i++ < clnt_cnt-1)
                clnt_socks[i] = clnt_socks[i+1];
            break;
        }
    }
    clnt_cnt--;
    pthread_mutex_unlock(&mutx);
    close(clnt_sock);
    return NULL;
}

void send_msg(char * msg, int len) // send to all
{
    int i;
    pthread_mutex_lock(&mutx);
    for (i = 0; i < clnt_cnt; i++)
        write(clnt_socks[i], msg, len);
    pthread_mutex_unlock(&mutx);
}
```

Multi-thread Client: chat_clnt.c

```
define BUF_SIZE 100
#define NAME_SIZE 20

void *send_msg(void * arg);
void *recv_msg(void * arg);
void error_handling(char * msg);

char name[NAME_SIZE]="[DEFAULT]";
char msg[BUF_SIZE];

int main(int argc, char *argv[])
{
    ...
    pthread_t snd_thread, rcv_thread;
    void * thread_return;
    ...

    if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
        error_handling("connect() error");

    pthread_create(&snd_thread, NULL, send_msg, (void*)&sock); 2개의 스레드 (send/recv)
    pthread_create(&rcv_thread, NULL, recv_msg, (void*)&sock);
    pthread_join(snd_thread, &thread_return);
    pthread_join(rcv_thread, &thread_return);
    close(sock);
    return 0;
}
```

```
void *send_msg(void * arg) // send thread main
{
    int sock = *((int*)arg);
    char name_msg[NAME_SIZE+BUF_SIZE];
    while (1)
    {
        fgets(msg, BUF_SIZE, stdin);
        if (!strcmp(msg, "q\n") || !strcmp(msg, "Q\n"))
        {
            close(sock);
            exit(0);
        }
        sprintf(name_msg, "%s %s", name, msg);
        write(sock, name_msg, strlen(name_msg));
    }
    return NULL;
}
```

```
void *recv_msg(void * arg) // read thread main
{
    int sock = *((int*)arg);
    char name_msg[NAME_SIZE+BUF_SIZE];
    int str_len;
    while (1)
    {
        str_len = read(sock, name_msg, NAME_SIZE+BUF_SIZE-1);
        if (str_len == -1)
            return (void*)-1;
        name_msg[str_len] = 0;
        fputs(name_msg, stdout);
    }
    return NULL;
}
```

실습#3

- Multi-thread를 활용한 File 전송 서버 구현
 - Day1의 file_server.c를 Multi-thread기반의 Multiple access server로 구현해보세요.
 - Client 프로그램은 Day1의 file_client.c를 그대로 사용해도 됩니다.
 - 서버가 동시에 여러 클라이언트를 서비스 할 수 있는지 검증해야합니다.

Network Programming (Day 4): roadmap

- Multi-Thread Programming #1
- Multi-Thread Programming #2
- **Web Server Implementation**

http 요청 -> 문자열 처리를 통해..
(dup, fileno 등 활용)

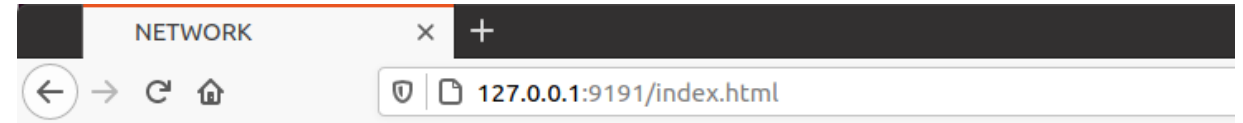
Simple Web Server

- Let's implement a simple web server using TCP socket
- Our web server can provide simple web page by client request

Server

```
yunmin@ym-ubuntu:~/Workspace$ ./webserv_linux 9191  
Connection Request : 127.0.0.1:52208  
Connection Request : 127.0.0.1:52210
```

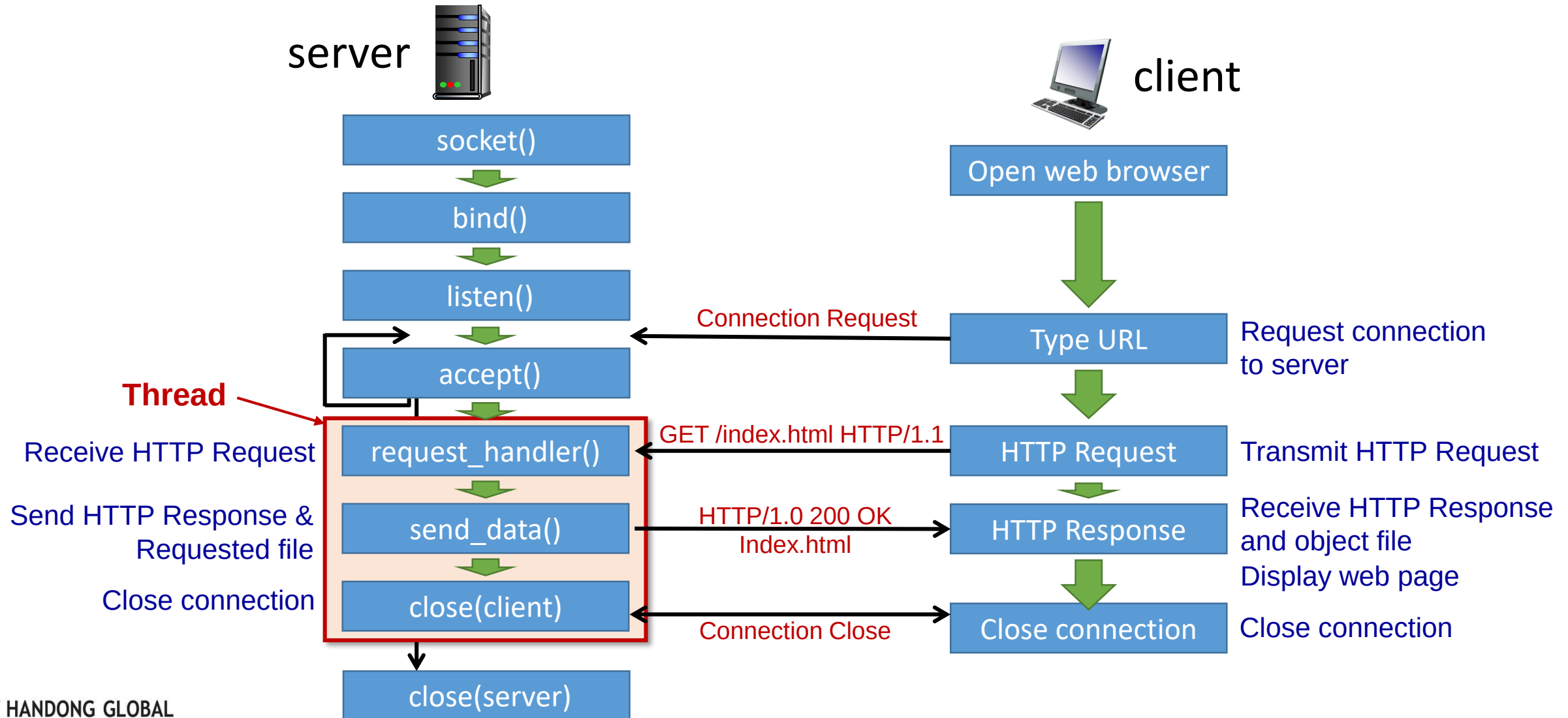
Client



Handong Global University!
Computer Network!
Socket Programming is Fun!

- We will implement a concurrent server
- For the client, we will use web browser instead of implementing client program

Function Calls for Simple Web Server



web_server.c: main()

```
int main(int argc, char *argv[])
{
    int serv_sock, clnt_sock;
    struct sockaddr_in serv_adr, clnt_adr;
    int clnt_adr_size;
    char buf[BUF_SIZE];
    pthread_t t_id;
    if (argc != 2) {
        printf("Usage : %s <port>\n", argv[0]);
        exit(1);
    }

    serv_sock=socket(PF_INET, SOCK_STREAM, 0);
    memset(&serv_adr, 0, sizeof(serv_adr));
    serv_adr.sin_family = AF_INET;
    serv_adr.sin_addr.s_addr=htonl(INADDR_ANY);
    serv_adr.sin_port = htons(atoi(argv[1]));
    if (bind(serv_sock, (struct sockaddr*)&serv_adr, sizeof(serv_adr)) == -1)
        error_handling("bind() error");
    if (listen(serv_sock, 20) == -1)
        error_handling("listen() error");
```

```
while(1)
{
    clnt_adr_size = sizeof(clnt_adr);
    clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_adr, &clnt_adr_size);
    printf("Connection Request : %s:%d\n",
        inet_ntoa(clnt_adr.sin_addr), ntohs(clnt_adr.sin_port));

    pthread_create(&t_id, NULL, request_handler, &clnt_sock);
    pthread_detach(t_id);
}
close(serv_sock);
return 0;
}
```

web_server.c: request_handler()

```
int* request_handler(void *arg)
{
    int clnt_sock = *((int*)arg);
    char req_line[SMALL_BUF];
    FILE* clnt_read;
    FILE* clnt_write;

    char method[10];
    char ct[15];
    char file_name[30];

    clnt_read = fdopen(clnt_sock, "r");
    clnt_write = fdopen(dup(clnt_sock), "w");
    fgets(req_line, SMALL_BUF, clnt_read);
    if (strstr(req_line, "HTTP/") == NULL)
    {
        send_error(clnt_write);
        fclose(clnt_read);
        fclose(clnt_write);
        return;
    }
}
```

```
    strcpy(method, strtok(req_line, " /"));
    strcpy(file_name, strtok(NULL, " /"));
    strcpy(ct, content_type(file_name));
    if (strcmp(method, "GET")!=0)
    {
        send_error(clnt_write);
        fclose(clnt_read);
        fclose(clnt_write);
        return;
    }

    fclose(clnt_read);
    send_data(clnt_write, ct, file_name);
}
```

web_server.c: send_data()

```
void send_data(FILE* fp, char* ct, char* file_name)
{
    char protocol[] = "HTTP/1.0 200 OK\r\n";
    char server[] = "Server:Linux Web Server \r\n";
    char cnt_len[SMALL_BUF];
    char cnt_type[SMALL_BUF];
    char buf[BUF_SIZE];
    FILE* send_file;
    int file_size;

    sprintf(cnt_type, "Content-type:%s\r\n\r\n", ct);
    send_file = fopen(file_name, "r");
    if (send_file == NULL)
    {
        send_error(fp);
        return;
    }

    printf("file_name: %s \n", file_name);
```

```
// GET file size and construct header for Content-length
fseek(send_file, 0, SEEK_END);
file_size = ftell(send_file);
sprintf(cnt_len, "Content-length:%d\r\n", file_size);
fseek(send_file, 0, SEEK_SET);

// Send header
fputs(protocol, fp);
fputs(server, fp);
fputs(cnt_len, fp);
fputs(cnt_type, fp);

// Send content
while (fgets(buf, BUF_SIZE, send_file) != NULL)
{
    fputs(buf, fp);
    fflush(fp);
}
fflush(fp);
fclose(fp);
}
```

web_server.c: content_type(), send_error()

```
char* content_type(char* file)
{
    char extension[SMALL_BUF];
    char file_name[SMALL_BUF];
    strcpy(file_name, file);
    strtok(file_name, ".");
    strcpy(extension, strtok(NULL, "."));

    if (!strcmp(extension, "html") || !strcmp(extension, "htm"))
        return "text/html";
    else
        return "text/plain";
}
```

```
void send_error(FILE* fp)
{
    char protocol[] = "HTTP/1.0 400 Bad Request\r\n";
    char server[] = "Server:Linux Web Server \r\n";
    char cnt_len[] = "Content-length:2048\r\n";
    char cnt_type[] = "Content-type:text/html\r\n\r\n";
    char content[] = "<html><head><title>NETWORK</title></head>"
                    "<body><font size=+5><br>Error! Please re-check the file name"
                    "and requet method"
                    "</font></body></html>";

    fputs(protocol, fp);
    fputs(server, fp);
    fputs(cnt_len, fp);
    fputs(cnt_type, fp);
    fflush(fp);
}
```

web_server.c: execution

■ Server

```
[gym@localhost practice12]$ ./web_server 9190  
Connection Request : 10.1.0.2:39614  
file_name: index.html
```

■ Client

```
[gym@localhost practice12]$ curl http://10.1.0.1:9190/index.html  
<html>  
<head><title>NETWORK</title></head>  
<body><font size=+5>  
Handong Global University! <br>  
Computer Network <br>  
Socket Programming is Fun!  
</font></body>  
</html>[gym@localhost practice12]$
```

Questions #4-1

1. Pthread에서 스레드를 생성하는 `pthread_create()` 함수의 인자들은 각각 어떤 역할을 하며, 스레드 생성 시 발생할 수 있는 오류는 무엇인가요?
2. `pthread_join()`을 사용하여 스레드 종료를 기다릴 때와 `pthread_detach()`를 사용할 때의 차이점은 무엇인가요?
3. Pthread API에서 제공하는 조건 변수(Conditional Variable)의 사용 목적은 무엇이며, 뮤텍스와 함께 사용할 때의 동작 방식을 설명해 보세요.
4. 스레드 간 데이터 공유의 장점과 단점을 시스템 자원 관점에서 분석해 보세요.
5. 멀티스레드 환경에서 전역 변수에 여러 스레드가 동시에 접근할 경우 발생할 수 있는 문제를 설명하고, 이를 방지하기 위한 해결책을 기술하세요.

Questions #4-2

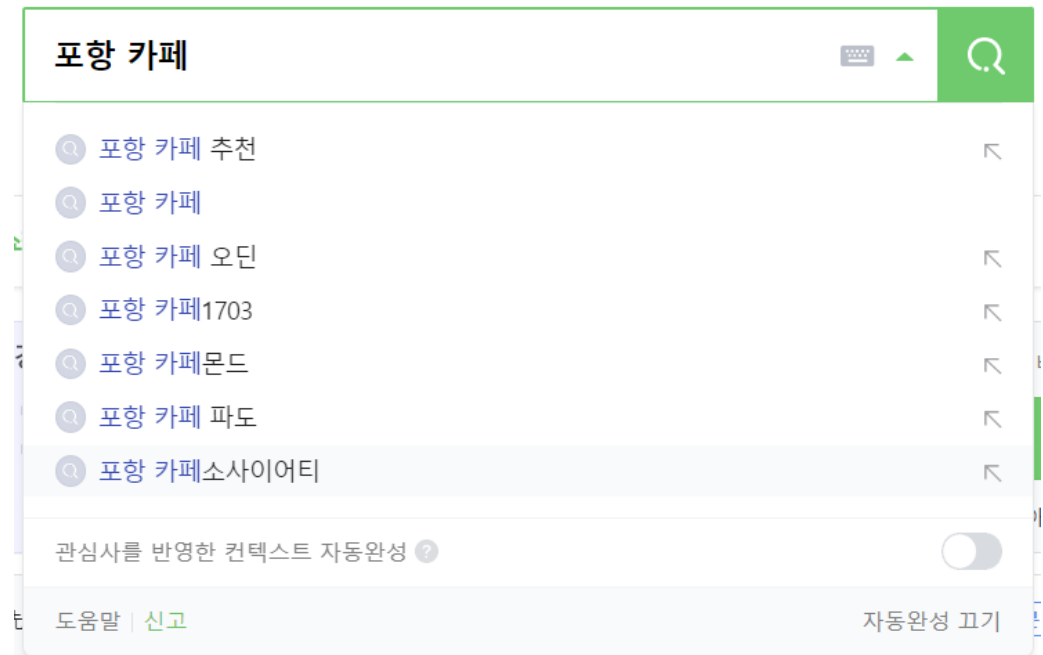
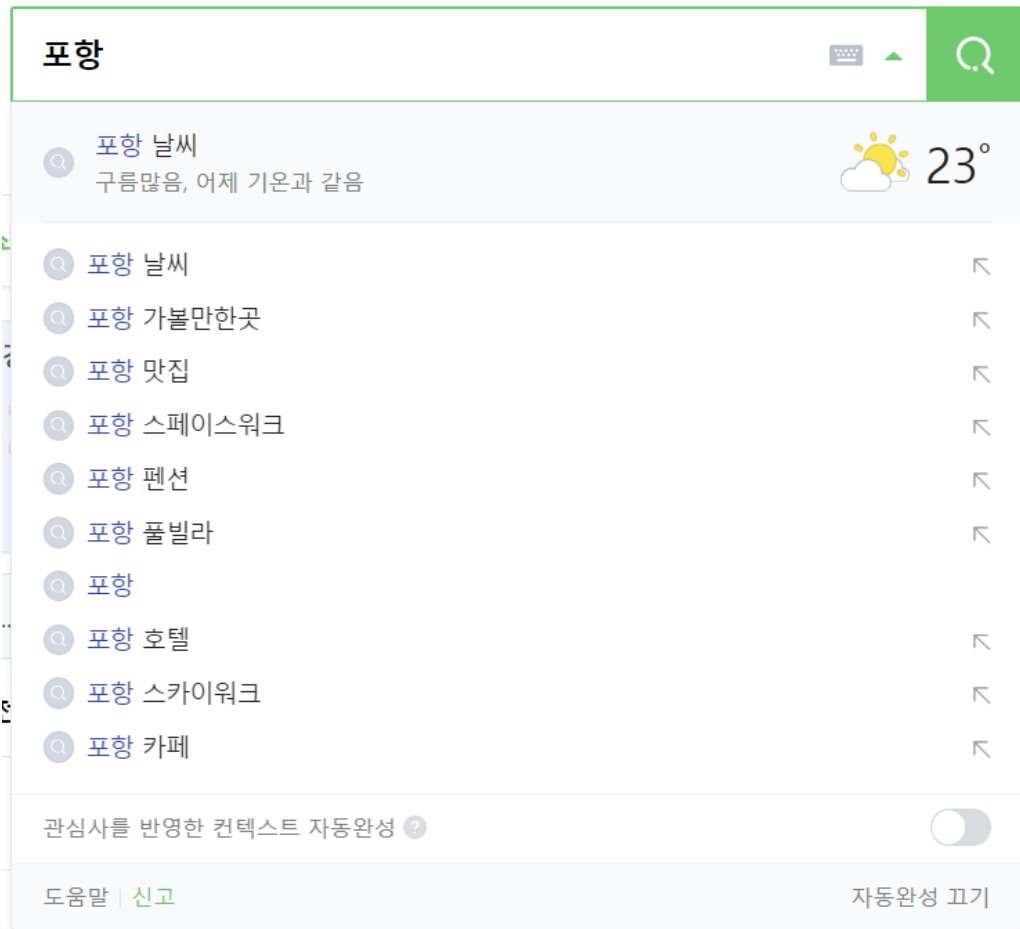
6. 뮉텍스(Mutex)와 세마포어(Semaphore)의 개념적 차이점은 무엇이며, 각각의 장단점은 어떤 상황에서 뚜렷하게 드러나나요?
7. mutex.c 예제에서 뮉텍스를 적용하기 전과 후의 실행 결과 차이가 발생하는 근본적인 이유를 설명하세요.
8. sem_post()와 sem_wait() 함수는 어떤 방식으로 스레드의 동기화를 보장하며, 데드락(deadlock)을 방지하기 위한 주의사항은 무엇인가요?
9. 세마포어 기반의 생산자-소비자 문제 해결 방식과 뮉텍스를 활용한 방식 간의 설계 철학 차이를 설명해 보세요.
10. 멀티스레드 기반의 채팅 서버에서 클라이언트 소켓 배열(clnt_socks[])을 안전하게 접근하기 위해 사용된 동기화 방식은 무엇이며, 그 필요성은 무엇인가요?

Questions #4-3

11. 서버의 `handle_clnt()` 함수에서 클라이언트 연결 종료 후 소켓 배열을 정리하는 방식의 한계와 개선점을 제시해 보세요.
12. 멀티스레드 서버가 대량의 동시 접속을 처리할 때 성능 저하를 유발할 수 있는 요소는 무엇이며, 이를 해결할 수 있는 고급 기법은 어떤 것이 있을까요?
13. 멀티스레드 기반의 웹 서버 구현에서 각 클라이언트 요청을 스레드로 처리하는 방식의 장단점을 설명하세요.
14. 웹 서버에서 클라이언트가 보낸 HTTP 요청을 파싱하고 처리하는 함수들(`request_handler`, `send_data`)의 구조를 설명하고, 이를 보안 측면에서 개선할 수 있는 방안을 제시해 보세요.

숙제 #4

■ 자동 완성 기능 기반 Search Engine 구현



숙제 #4

- 자동 완성 기능 기반 Search Engine 구현
 - 서버의 Listen 포트 번호와 검색어 데이터베이스 파일을 command line argument 로 넣어 프로그램을 실행한다.

```
$ ./search_server 9090 data.txt
```

- 파일로부터 검색어와 검색어의 검색 횟수를 읽어 온다.

data.txt 의 예

Pohang	30000
Pohang Weather	40000
Pohang Café	80000
Pohang Restaurant	50000
Pohang Hotel	10000

숙제 #4

■ 자동 완성 기능 기반 Search Engine 구현

- 클라이언트에서 검색어를 입력하면 서버는 연관 검색어를 검색 횟수에 대해 내림차순 정렬하여 표시한다. (Trie Data structure를 사용해도 됨)
- 클라이언트에서 검색어 입력 시 검색어를 문자별로 인식하여 Enter를 치지 않더라도 실시간으로 검색어가 출력되게 한다. 연관검색어는 최대 10개까지만 출력
- 연관 검색어 출력 시 검색어에 해당되는 부분은 임의의 색깔을 사용한다.

```
$ ./ search_client 192.168.10.2 9090
```

```
Search Word: Pohang
```

```
-----  
Pohang Café
```

```
Pohang Restaurant
```

```
Pohang Weather
```

```
Pohang
```

```
Pohang Hotel
```

alphabetical order

콘솔 출력 제어

<https://rottk.tistory.com/entry/%EC%BD%98%EC%86%94-%EB%8B%A4%EB%A3%A8%EA%B8%B0-Escape-Sequence-C>

출력 색상 변경

https://velog.io/@chez_bono/C-%EC%BD%98%EC%86%94%EC%97%90-%EC%B6%9C%EB%A0%A5%EB%90%98%EB%8A%94-%ED%85%8D%EC%8A%A4%ED%8A%B8-%EC%83%89%EC%83%81-%EB%B3%80%EA%B2%BD

숙제 #4

- 자동 완성 기능 기반 Search Engine 구현
 - 검색어가 제일 처음에 나오는 경우 뿐만 아니라 **중간**에 나오거나 **끝**에 나오는 경우도 고려한다.
 - 서버는 **Multi-Thread**로 구현한다.
 - Makefile로 컴파일 필요

